Stockade

Full Project Status Report — What's Been Done, Outcomes, and Bottlenecks

Covers both active tracks: the single-vehicle dash-cam collision-risk pipeline (Phases 1–2) and the cooperative multi-agent perception track (V2X-Sim), with every key technical detail, measured result, and open bottleneck.

Prepared: October 11, 2026

Project folder: Stockade (local, no version control)

Hardware: single machine, NVIDIA RTX 5060 Ti (16GB), 16 CPU cores

TRACK A

Dash-cam Collision-Risk Pipeline (Phase 1 – 2)

Phase 1 — Detection & Tracking (`track_vehicles.py`)

Status: complete. YOLOv8 + ByteTrack over dash-cam clips, producing an annotated video (boxes + persistent track IDs) and a per-clip trajectory CSV per vehicle.

- Ran across the **full DAD dataset: 1,750 clips** (620 labeled crash / 1,129 labeled normal, plus 1 corrupt file).
- One file, `training/negative/000828.mp4`, failed with "moov atom not found" (genuinely corrupted/truncated source file). The script originally had no error handling, so this crashed the entire batch partway through (at clip 1,293/1,750).
- **Fix applied:** hardened the script to (a) skip clips whose output CSV already exists, so it can resume idempotently, and (b) catch per-clip exceptions and log-and-continue instead of crashing the whole run. Re-running then completed the remaining clips in minutes, correctly skipping the one truly corrupt file.
- Final result: **1,749 / 1,750 clips** successfully processed.

Phase 2 — Risk Scoring (`score_risk.py`)

Status: functional baseline, not yet closed out. A deliberately transparent rule-based scorer (no ML) using two geometric signals — closing-speed (windowed regression slope of box area, a time-to-collision proxy) and lateral weaving (variance in horizontal position).

Version history

Version	Change	Result
v1	Single frame-to-frame diff for closing speed	Very low recall — most real crashes scored near zero
v2	Windowed regression slope, box area instead of height, confidence filtering, proximity gate	52% recall but high false-alarm rate; a persistence-gate bug silently zeroed the core "closing fast" signal
v3	Fixed the persistence-gate bug (it was mathematically impossible to trigger), dropped an inert "peer-relative" check, retuned thresholds for precision	See full-scale results below

Full-scale evaluation (v3, all 1,749 processed DAD clips: 620 crash / 1,129 normal)

Threshold	Recall	Precision	False-alarm rate
0.3	56% (350/620 crashes caught)	43%	42%
0.5	38% (236/620 crashes caught)	44%	27%
0.7	12% (75/620 crashes caught)	45%	8%

> **Finding:** an earlier evaluation on a small 48-clip hand-picked sample had shown ~60% precision at threshold 0.5. Running the exact same scorer across the full labeled set revealed the real number is **~44%** — the small sample's near-even 23/23 crash/normal split was not representative of the dataset's real 620:1,129 imbalance. The full-scale numbers above are the trustworthy ones.

Bottleneck

> **Two hand-picked geometric signals have a hard accuracy ceiling.** At the most balanced threshold (0.5), the scorer still misses 62% of real crashes, or at a looser threshold, false-alarms on 42% of normal clips. This is a structural limit of OR-ing two fixed-threshold rules, not a tuning problem — the signals can't capture how closing-speed, weaving, and proximity interact. The agreed next step is a trained classifier (logistic regression, then gradient-boosted trees) on the same per-frame features, using DAD's built-in training/testing split for a trustworthy train/test evaluation. **Not yet started.**

TRACK B

Cooperative Multi-Agent Perception (V2X-Sim)

Scope decision

Four V2X/cooperative-infrastructure datasets were sitting unextracted (~15GB total, one at 77GB uncompressed). Investigation showed **none contain crash/risk labels** — they're built for 3D object detection and trajectory forecasting between a vehicle and roadside infrastructure, not for the Phase 2 risk classifier. Decision: pursue this as a separate cooperative-perception track, starting with **V2X-Sim-det-001** (77GB uncompressed, simulated multi-agent LiDAR BEV data, already preprocessed into training-ready tensors).

Dataset

- **V2X-Sim-det-001**: 6 simulated agents (agent0 = roadside unit / RSU, agent1–5 = vehicles), split into **8,000 train / 1,000 val / 1,000 test** scene-frames.
- Each agent's sample: a **256×256×13 BEV voxel grid** (64m×64m area, 0.25m resolution).
- Training used agent1–5 only (the "no RSU" baseline configuration).

Engineering: getting a 2020-era benchmark running on modern hardware

Rather than build a cooperative-detection model from scratch, we reused the official `coperception` SDK (the toolkit behind the V2X-Sim/DiscoNet benchmark papers), which ships ready-made baselines (lowerbound, upperbound, DiscoNet, When2com, V2VNet, and others) built specifically for this exact data format. Getting it running on this machine (Python 3.11, Windows, RTX 5060 Ti, CUDA 12.8) required several fixes to a codebase built for Python 3.7 on Linux circa 2020–2022:

Problem	Fix
`pip install coperception` failed — modern setuptools' isolated build no longer ships `pkg_resources`	Cloned the source repo directly and ran it via `PYTHONPATH` instead of a pip install
Package bundles a compiled `mapping` extension built only for Python 3.7 (`cp37-win_amd64.pyd`) — fails to load on Python 3.11	Confirmed it's only used by raw-LiDAR preprocessing (not needed since our data is already preprocessed); patched the one import to skip gracefully
`nuscenes-devkit==1.0.9` pulled in pandas, motmetrics, fire, jupyter, and an old `Pillow<=6.2.1` pin — just to reach one small geometry class (`Box.corners()`)	Reimplemented that one class locally (~60 lines, copied faithfully from the original), skipping the entire legacy dependency chain
`mmcv` (version/CUDA-pinned, unbuildable against this machine's bleeding-edge torch+CUDA combo) needed only for one logging helper (`print_log`) in the mAP eval code	Stubbed the same way — a 10-line local replacement
Windows `multiprocessing.Manager()` (used for the dataset's internal sample cache) requires the entry script to guard with `if __name__ == "__main__":`	Added the guard to our smoke-test scripts; caused one hung-process incident, cleaned up via `Stop-Process`
Upstream training script crashes immediately if a certain checkpoint-check directory doesn't already exist	Pre-created the expected directory as a workaround
Default pip install gave CPU-only PyTorch despite an idle RTX 5060 Ti	Reinstalled PyTorch from the CUDA 12.8 (cu128) wheel index; confirmed GPU detection

Validation before committing to a long run

- Single-batch smoke test: dataset loading → model forward → backward step, confirmed working end-to-end on real data on the GPU.
- Data-loading speed test: **0 workers → 1.07 it/s** (~31 min/epoch) vs. **6 workers → ~1.6 it/s** (~20–22 min/epoch). Only a ~50% speedup from 6x the worker processes — GPU compute and per-sample voxel densification share the bottleneck with disk I/O, not disk I/O alone.

Full training run

Setting	Value
Model	`FaFNet` , `lowerbound` config (no cross-agent cooperation)
Epochs	100
Batch size	4 scene-frames (×5 agents = 20 effective samples/step)
Optimizer / LR	Adam, lr 0.001, `MultiStepLR` decay (×0.5) at epochs [50, 100, 150, 200]
Data workers	6
Total wall time	**~36 hours** (Oct 7 22:12 → Oct 9 10:10), steady ~21–22 min/epoch throughout
Checkpoints	Saved every epoch (100 total, ~90MB each)

Loss trajectory

	Classification loss	Localization loss
Epoch 1 (start)	~19,300	~41,000
Epoch 33	~90	~920
Epoch 100 (final)	~28	~413

Loss dropped roughly two orders of magnitude overall, but **flattened out around epoch 30–35** — the epochs from 95–100 were all in the same 420–430 average-loss band as epoch 33.

Accuracy (mAP, measured via the toolkit's own evaluation script on the held-out 1,000-scene val split)

Checkpoint	mAP@0.5	mAP@0.7
Epoch 33	63.2%	60.6%
Epoch 100 (final)	**64.1%**	**60.9%**
Gain over 67 epochs	+0.9 pt	+0.3 pt

Per-agent spread at the final checkpoint: 58.8%–69.5% (mAP@0.5). The marginal gain over two-thirds of the training run confirms the loss plateau corresponds to a real accuracy plateau, not just noise.

Bottlenecks

> **The 100-epoch run only passed 1 of the 4 scheduled LR-decay steps.** The scheduler is configured for milestones at epochs [50, 100, 150, 200] — i.e. built for a 200-epoch recipe. Stopping at exactly epoch 100 means the run only ever saw the first LR halving (0.001 → 0.0005) and stopped right at the second one. Part of the

> apparent "plateau" may be the learning rate sitting too high for a schedule designed to run twice as long, rather than genuine convergence.

> **lowerbound's accuracy ceiling is structural, not a training problem.** Each agent only sees its own LiDAR, with zero sharing between vehicles. A car occluded from one agent's view or outside its sensor range simply cannot be detected, no matter how long training runs. 64.1%/60.9% mAP is a plausible floor for this constraint — closing the gap requires an actual fusion model (DiscoNet, V2VNet, etc.), not more lowerbound epochs.

Decided next step (not yet started)

Train **DiscoNet** (feature-fusion across all 5 agents) on the identical dataset/split/schedule, then run the same mAP evaluation on the same held-out val split for a direct, apples-to-apples comparison against the 64.1% / 60.9% no-cooperation floor established here. This is the actual test of how much accuracy is recoverable through cooperation.

Cross-cutting notes

- **No version control.** The project folder is not a git repository — there's no commit history backing any of the script changes (v1→v2→v3 scorer, track_vehicles.py hardening) beyond what's recorded in code comments and this report.
- **Single machine, single GPU.** Every long-running job in this report (full DAD tracking, full-scale Phase 2 eval, 100-epoch cooperative-perception training) ran sequentially on one RTX 5060 Ti. A DiscoNet run will add another ~36 hours on top of everything already spent.
- **Environment now stabilized.** Despite the dependency fights documented above, there is now a working Windows-native Python environment (`stockade-env`) with GPU-enabled PyTorch, Ultralytics, and a patched local copy of `coperception` (`reference_coperception/`) that runs cleanly for both training and evaluation.